Faith James

Dr. Huang

COSC 647

9 November 2026

Database Security in PostgreSQL: Auditing, Logging, Alerts, and Encryption

This report examines the free, open-source security features of PostgreSQL, focusing on the PostgreSQL Audit Extension (pgAudit). It covers the four areas named in the lab, tracing, auditing, alerts, and encryption, and describes a hands-on test of each.

Introduction to the Features

PostgreSQL's built-in logging records statements as unstructured text that is hard to feed into compliance tools (JusDB). pgAudit adds session auditing, which records whole classes of statements, and object auditing, which records activity on chosen tables, with each entry beginning with AUDIT: (pgAudit, "pgaudit/pgaudit"). Row-level security limits each user to their own rows (PostgreSQL Global Development Group, "Row Security"). Triggers can call NOTIFY to alert any client running LISTEN (PostgreSQL Global Development Group, "NOTIFY"). The pgcrypto extension encrypts columns and hashes passwords in SQL (PostgreSQL Global Development Group, "F.26. pgcrypto"), and SSL/TLS protects data on the network (PostgreSQL Global Development Group, "Secure TCP/IP Connections").

Hands-On Testing

I installed PostgreSQL 16.15 on Ubuntu Linux with the free pgAudit 16.0 package, added pgAudit to `shared_preload_libraries`, enabled SSL, and restarted the server. I then created a database named `labdb` and installed both extensions (fig. 1).

```
labdb=# \dx
                    List of installed extensions
   Name   | Version |   Schema   |           Description
----------+---------+------------+--------------------------------
 pgaudit  | 16.0    | public     | provides auditing functionality
 pgcrypto | 1.3     | public     | cryptographic functions
 plpgsql  | 1.0     | pg_catalog | PL/pgSQL procedural language
(3 rows)
```

Fig. 1. The \dx command listing the installed pgaudit and pgcrypto extensions.

With `pgaudit.log` set to `write, ddl`, I created an *employees* table, inserted three rows, updated a salary, and read the table. The log recorded the CREATE TABLE, INSERT, and UPDATE but not the SELECT, as configured (fig. 2). For object auditing, I granted an *auditor* role SELECT on only the *salary* column; a query of names left no entry, while a query of salaries did (fig. 3).

```
2026-11-09 00:26:57.524 UTC [1082] postgres@labdb LOG:  AUDIT: SESSION,1,1,DDL,CREATE SEQUENCE,SEQUENCE,public.employees_id_seq,"CREATE TABLE employees (id serial PRIMARY KEY, name text, dept text, salary numeric, owner_name text);",<not logged>
2026-11-09 00:26:57.524 UTC [1082] postgres@labdb LOG:  AUDIT: SESSION,1,1,DDL,CREATE TABLE,TABLE,public.employees,"CREATE TABLE employees (id serial PRIMARY KEY, name text, dept text, salary numeric, owner_name text);",<not logged>
2026-11-09 00:26:57.524 UTC [1082] postgres@labdb LOG:  AUDIT: SESSION,1,1,DDL,CREATE INDEX,INDEX,public.employees_pkey,"CREATE TABLE employees (id serial PRIMARY KEY, name text, dept text, salary numeric, owner_name text);",<not logged>
2026-11-09 00:26:57.524 UTC [1082] postgres@labdb LOG:  AUDIT: SESSION,1,1,DDL,ALTER SEQUENCE,SEQUENCE,public.employees_id_seq,"CREATE TABLE employees (id serial PRIMARY KEY, name text, dept text, salary numeric, owner_name text);",<not logged>
2026-11-09 00:26:57.525 UTC [1082] postgres@labdb LOG:  AUDIT: SESSION,2,1,WRITE,INSERT,TABLE,public.employees,"INSERT INTO employees (name, dept, salary, owner_name) VALUES ('Alice Chen','Sales',62000,'alice'), ('Bob Diaz','IT',71000,'bob'), ('Carla Ruiz','HR',58000,'carla');",<not logged>
2026-11-09 00:26:57.526 UTC [1082] postgres@labdb LOG:  AUDIT: SESSION,3,1,WRITE,UPDATE,TABLE,public.employees,UPDATE employees SET salary = 75000 WHERE name = 'Bob Diaz';,<not logged>
```

Fig. 2. Session auditing: AUDIT entries written to the server log for the CREATE TABLE, INSERT, and UPDATE run as hr_admin. The SELECT does not appear because the READ class was not enabled.

```
labdb=# SELECT name, salary FROM employees WHERE dept = 'IT';
   name   | salary
----------+--------
 Bob Diaz |  75000
(1 row)
```

```
2026-11-09 00:26:59.101 UTC [1090] postgres@labdb LOG:  AUDIT: OBJECT,1,1,READ,SELECT,TABLE,public.employees,"SELECT name, salary FROM employees WHERE dept = 'IT';",<not logged>
```

Fig. 3. Object auditing: the query that reads the audited salary column (top) and the OBJECT audit entry it produced (bottom).

Next, a row-level security policy let each user see only rows where *owner_name* matched their login, so the same query returned different results for alice and bob (fig. 4). For alerts, I wrote a trigger that calls `pg_notify` when a salary rises by more than ten percent. A monitoring session running LISTEN received an alert when Alice Chen's salary jumped from 62,000 to 90,000, while a small raise was ignored (fig. 5).

```
labdb=# SET ROLE alice;
SET
labdb=# SELECT name, dept, salary FROM employees;
    name    | dept  | salary
------------+-------+--------
 Alice Chen | Sales |  62000
(1 row)

labdb=# SET ROLE bob;
SET
labdb=# SELECT name, dept, salary FROM employees;
   name   | dept | salary
----------+------+--------
 Bob Diaz | IT   |  75000
(1 row)
```

Fig. 4. Row-level security: the same query returns only Alice's row for alice and only Bob's row for bob.

```
Asynchronous notification "security_alerts" with payload "ALERT: Alice Chen salary changed 62000 -> 90000 by hr_ad
min" received from server process with PID 1110.
```

Fig. 5. The alert received by the monitoring session after hr_admin raised Alice Chen's salary from 62,000 to 90,000.

Finally, I encrypted a sample Social Security number with AES-256 using `pgp_sym_encrypt`. The stored value was unreadable, the correct key decrypted it, and a wrong key was rejected; a bcrypt password hash also passed and failed login checks correctly (fig. 6). A connection with `sslmode=require` used TLS 1.3 with 256-bit encryption (fig. 7).

```
labdb=# SELECT name, left(encode(ssn_enc, 'hex'), 40) || '...' AS ssn_as_stored FROM customers;
   name   |               ssn_as_stored
----------+--------------------------------------------
 Dana Lee | c30d04090302aa0b0e0279d18eba7fd23c0151cf...
(1 row)

labdb=# SELECT name, pgp_sym_decrypt(ssn_enc, 'S3cretKey!') AS ssn FROM customers;
   name   |     ssn
----------+-------------
 Dana Lee | 123-45-6789
(1 row)

labdb=# SELECT pgp_sym_decrypt(ssn_enc, 'WrongKey') FROM customers;
psql:7_crypto.sql:5: ERROR:  Wrong key or corrupt data
```

```
labdb=# SELECT username, pw_hash FROM app_users;
 username |                           pw_hash
----------+--------------------------------------------------------------
 dana     | $2a$08$EBqpkBfG56FH2KHlQLIPpOF0jV2Sx38UEZ1YG3gEQFgbR3Mgby6Oa
(1 row)

labdb=# SELECT username, pw_hash = crypt('Tr0ub4dor', pw_hash) AS login_ok FROM app_users;
 username | login_ok
----------+----------
 dana     | t
(1 row)

labdb=# SELECT username, pw_hash = crypt('guess123', pw_hash) AS login_ok FROM app_users;
 username | login_ok
----------+----------
 dana     | f
(1 row)
```

Fig. 6. pgcrypto: the encrypted SSN as stored, decrypted with the correct key, and rejected with a wrong key (top); a bcrypt password hash and login checks (bottom).

```
labdb=# \conninfo
You are connected to database "labdb" as user "postgres" on host "localhost" (address "127.0.0.1") at port "5433".
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
labdb=# SELECT pid, ssl, version, cipher, bits FROM pg_stat_ssl WHERE pid = pg_backend_pid();
 pid  | ssl | version |         cipher         | bits
------+-----+---------+------------------------+------
 1116 | t   | TLSv1.3 | TLS_AES_256_GCM_SHA384 |  256
(1 row)
```

Fig. 7. TLS 1.3 connection details reported by \conninfo and the pg_stat_ssl view.

User Experience

The tools are powerful but demanding. Everything is configured through SQL and text files, with no graphical console for audits or alerts. pgAudit requires a server restart to install, although later setting changes only need `pg_reload_conf()`, and cloud platforms such as Supabase reduce setup to a toggle (Supabase). Audit logs are easy to filter, but volume grows quickly; one CREATE TABLE produced four audit lines, and AWS advises monitoring storage (Amazon Web Services). pgcrypto uses familiar SQL but leaves key management to the user (Neon).

Pros and Cons

A key strength is that pgAudit keeps passwords out of the log (Amazon Web Services), unlike built-in logging (Microsoft). In my test, built-in logging recorded a new password in plain text while pgAudit replaced it with <REDACTED> (fig. 8).

```
2026-11-09 00:27:05.411 UTC [1124] postgres@labdb LOG:  statement: ALTER ROLE hr_admin PASSWORD 'N3wSecret!';
2026-11-09 00:27:06.991 UTC [1132] postgres@labdb LOG:  AUDIT: SESSION,1,1,ROLE,ALTER ROLE,,,ALTER ROLE hr_admin P
ASSWORD <REDACTED>,<not logged>
```

Fig. 8. The same ALTER ROLE command logged by built-in statement logging (password visible) and by pgAudit (password redacted).

Pros:

- Every feature is free and supported by major cloud providers.
- Audit entries are consistent and filterable at the session or object level.
- Row-level security, NOTIFY, pgcrypto, and TLS are built in.

Cons:

- Setup requires configuration files, a restart, and no graphical interface.
- pgAudit cannot reliably audit superusers, and entries may be lost in a crash (pgAudit, "pgaudit/pgaudit").
- NOTIFY reaches only connected clients, and encrypted columns cannot be searched efficiently.

Conclusion

PostgreSQL with pgAudit and pgcrypto provides a complete, free set of database security controls. In this lab, each feature worked as documented. These tools suit organizations willing to invest setup time and manage log volume, but the lack of a graphical interface is a barrier for beginners.

Works Cited

Amazon Web Services. "Using pgAudit to Log Database Activity." *Amazon RDS User Guide*, docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Appendix.PostgreSQL.CommonDBATasks.pgaudit.html. Accessed 8 Nov. 2026.

JusDB. "pgAudit: PostgreSQL Audit Logging for SOC2, PCI-DSS, and HIPAA Compliance." *JusDB*, 5 Mar. 2026, www.jusdb.com/blog/pgaudit-postgresql-audit-logging-compliance.

Microsoft. "Audit Logging in Azure Database for PostgreSQL Flexible Server." *Microsoft Learn*, 16 July 2026, learn.microsoft.com/en-us/azure/postgresql/security/security-audit.

Neon. "The pgcrypto Extension." *Neon Docs*, 5 June 2026, neon.com/docs/extensions/pgcrypto.

pgAudit. "pgaudit/pgaudit: PostgreSQL Audit Extension." *GitHub*, github.com/pgaudit/pgaudit. Accessed 8 Nov. 2026.

PostgreSQL Global Development Group. "F.26. pgcrypto—Cryptographic Functions." *PostgreSQL 18 Documentation*, www.postgresql.org/docs/current/pgcrypto.html. Accessed 8 Nov. 2026.

---. "NOTIFY." *PostgreSQL 18 Documentation*, www.postgresql.org/docs/current/sql-notify.html. Accessed 8 Nov. 2026.

---. "Row Security Policies." *PostgreSQL 18 Documentation*, www.postgresql.org/docs/current/ddl-rowsecurity.html. Accessed 8 Nov. 2026.

---. "Secure TCP/IP Connections with SSL." *PostgreSQL 18 Documentation*, www.postgresql.org/docs/current/ssl-tcp.html. Accessed 9 Nov. 2026.

Supabase. "PGAudit: Postgres Auditing." *Supabase Docs*, 5 Oct. 2026, supabase.com/docs/guides/database/extensions/pgaudit.